



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

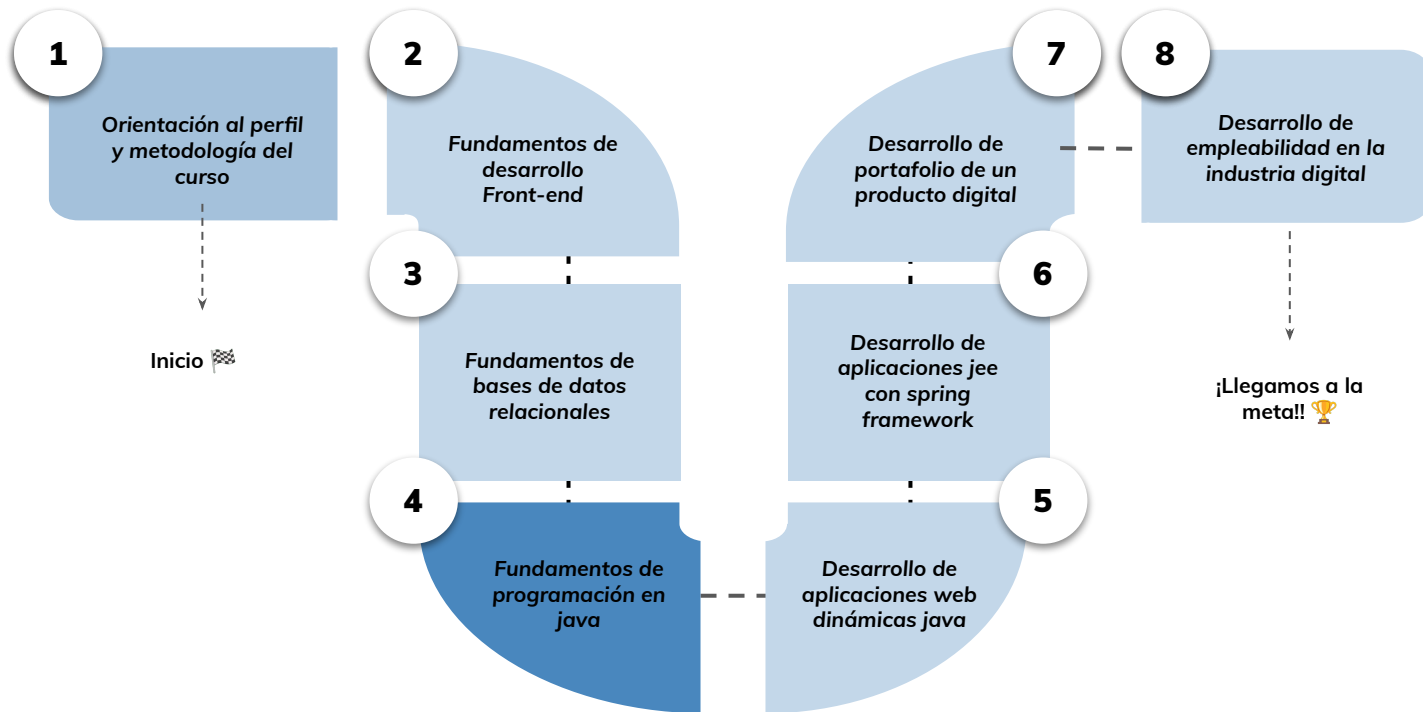


› Polimorfismo y principios básicos de diseño - Parte I

Aprendizaje Esperado: Utilizar principios básicos de diseño orientado a objetos para la implementación de una pieza de software acorde al lenguaje java para resolver un problema de baja complejidad.

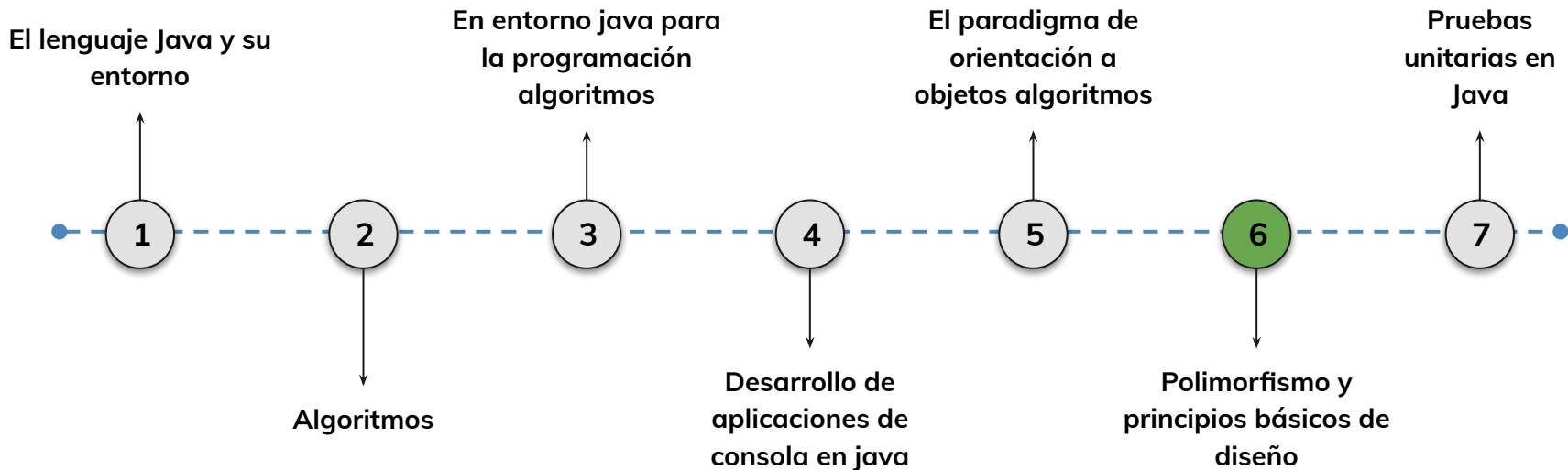
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

6.

Polimorfismo y principios básicos de diseño

Aprenderemos a aplicar principios fundamentales de diseño orientado a objetos utilizando herencia, polimorfismo, clases abstractas e interfaces en el lenguaje Java.

Herencia y Constructores

Acceso a atributos y métodos heredados

Uso de `super()` en constructores

Polimorfismo y Clases Abstractas

Sobreescritura de métodos y anotación `@Override`

Declaración de clases y métodos abstractos



Objetivos de aprendizaje

¿Qué aprenderás?



- Reconocer cómo se implementa la herencia en Java utilizando extends y super.
- Comprender el uso de constructores y atributos protegidos en jerarquías de clases.
- Aplicar el principio de polimorfismo mediante herencia y uso de interfaces.
- Diferenciar clases abstractas, métodos abstractos e interfaces en Java.
- Implementar código orientado a objetos reutilizable y escalable con principios de diseño básicos.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Reconocimos la importancia del encapsulamiento de información a través de la implementación de métodos accedadores y mutadores (get y set)
- Reconocimos las relaciones de agregación y composición.
- Comprendimos la implementación de las relaciones entre clases
- Comprendimos la utilidad de los diagramas de clase para la consecuente codificación

» Herencia

< > Herencia

¿Qué es y para qué se utiliza?

La herencia es una de las características fundamentales de la Programación Orientada a Objetos.

Mediante la herencia podemos **definir una clase a partir de otra ya existente** .

La clase nueva se llama clase **hija** o **subclase** y la clase existente se llama clase **padre** o **superclase** .

En esta relación, la frase “Un objeto **es un-tipo-de** una superclase” debe tener sentido, por ejemplo: un perro **es un tipo** de animal, o también, una heladera **es un tipo** de electrodoméstico.

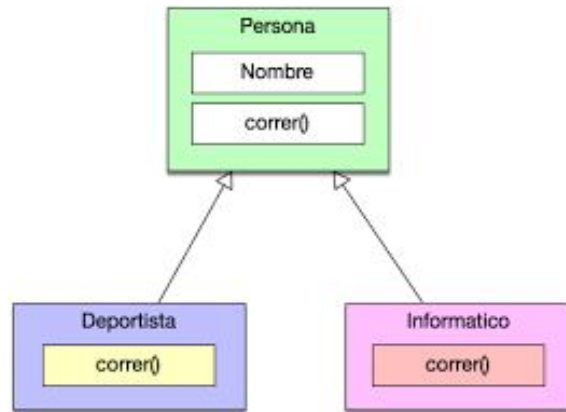


< > Herencia

La herencia apoya el concepto de "**reutilización**", es decir, cuando deseamos crear una nueva clase y ya existe una clase que incluye parte del código que queremos, podemos reutilizar los campos y métodos de la clase existente.

La manera de usar herencia es a través de la palabra **extends**.

```
public class SubClase extends SuperClase {  
    // Contenido de la clase ;  
}
```



➤ Herencia y atributos

< > Herencia: Atributos

En la **superclase** los atributos están creados con el modificador de acceso **protected** . Esto es porque el modificador de acceso `protected` permite que las subclases puedan acceder a los atributos de la superclase **sin la necesidad de getters y setters** .

Todos los atributos de la superclase se heredan a la clase hija.

Una subclase puede acceder a los miembros **públicos** y **protegidos** de la clase padre como si fuesen miembros propios.

```
public class Vehiculo {  
    protected int id;  
    protected String tipo;  
    protected String marca;  
}  
  
public class Coche extends Vehiculo{  
    private String color;  
}
```



LIVE CODING

Practicando con Herencia:

Veamos un ejemplo simple de dos clases relacionadas a través de la herencia.

1. *Crear un programa donde se ejecute la declaración de una superclase Persona con atributos nombre y rut. Luego, declarar una clase hija Empleado con el atributo “cargo”. También declarar una clase hija Cliente con atributo “tipo” (titular o adjunto).*

Tiempo: 20 minutos

➤ Herencia y constructores

< > Herencia: Constructores

Los constructores no se heredan de manera directa. Todos los constructores definidos en una superclase **pueden ser usados** desde constructores de las subclases **a través de la palabra reservada super**.

La palabra clave **super** es la que permite elegir qué constructor quiero usar.

Si la superclase tiene definido el constructor vacío y no colocamos una llamada explícita `super`, se llamará el constructor vacío de la superclase.

```
public class Coche extends Vehiculo{  
    private String color;  
  
    public Coche(String tipo, String marca, int id, String color) {  
        super(tipo, marca, id);  
  
        this.color = color;  
    }  
}
```

< > Herencia: Constructores

La palabra clave **super** sirve para hacer referencia o llamar a los atributos, métodos y constructores de la superclase en las clases hijas.

super.atributoClasePadre;

super.metodoClasePadre;

```
public class Coche extends Vehiculo{  
    private String color;  
    public Coche(String tipo, String marca, int id, String color) {  
        super(tipo, marca, id);  
        this.color = color;  
    }  
}
```



LIVE CODING

Practicando con Herencia:

Veamos un ejemplo simple de dos clases relacionadas a través de la herencia.

1. *Dada la clase Cuenta, crear una subclase CuentaCorriente que herede el constructor usando super(). Crear un objeto CuentaCorriente y verificar el constructor.*

Tiempo: 20 minutos



Ejercicio N° 1

Veterinaria 1.0

Veterinaria 1.0

Consigna 📝:

Dada una clase Animal con atributos nombre y peso, crear una subclase Perro que herede los atributos y además incluya la raza.

Luego, crear un objeto Perro utilizando el constructor super() e imprimir todos sus valores por pantalla.

Tiempo 🕒: 20 minutos

➤ Herencia y métodos

< > Herencia y métodos

Repasemos algunas características de la Herencia:

- Una subclase hereda de la superclase sus componentes (atributos y **métodos**).
- Los constructores no se heredan.
- Una subclase puede acceder a los miembros públicos y protegidos de la superclase como si fuesen miembros propios.
- Una subclase puede añadir a los miembros heredados, sus propios atributos y métodos (extender la funcionalidad de la clase).
- **También puede modificar los métodos heredados** .
- Una subclase puede, a su vez, ser una superclase, dando lugar a una **jerarquía de clases**.



< > Herencia y métodos

¿Cómo se comportan los métodos en la Herencia?: Sobreescritura

Los métodos heredados pueden ser redefinidos en las clases hijas. Este mecanismo se denomina sobreescritura. La sobreescritura permite a las clases hijas utilizar un método definido en la superclase.

Una subclase sobreescrive un método de su superclase cuando define un método con las mismas características (nombre, número y tipo de argumentos) que el método de la superclase. Las subclases emplean la sobreescritura de métodos la mayoría de las veces para agregar o modificar la funcionalidad del método heredado de la clase padre.



< > Herencia y métodos

Sobreescritura: @Override

La sobreescritura permite que las clases hijas sumen sus métodos en torno al funcionamiento, y esto se logra escribiendo la anotación **@Override** arriba del método que queremos sobreescribir. El método **debe llamarse igual** en la subclase como en la superclase.

```
@Override //indica que se modifica un método heredado
public void leer(){
}
```

Cuando en una subclase se redefine un método de una superclase, se oculta el método de la clase padre y todas las sobrecargas del mismo en la clase padre. Por eso para ejecutar el método leer() se debe escribir **super.leer();**



LIVE CODING

¡Probando los métodos!

*Vamos a trabajar sobre la clase **Cuenta** con los métodos **depositar()** y **retirar()**.*

*Luego, crearemos las **subclases** CuentaPesoCL y CuentaUSD que implementarán estos métodos realizando el cambio de moneda correspondiente.*

Vamos a mostrar por pantalla los resultados para verificar.

Tiempo: 30 minutos



Ejercicio N°

Veterinaria 2.0

Veterinaria 2.0

Consigna 📝:

Seguiremos trabajando sobre las clases Animal y Perro creadas en la clase anterior!

1- Declara los siguientes métodos en la superclase Animal: comer(), dormir() y emitirSonido().

2- La subclase Perro tendrá el siguiente comportamiento:

comer(): llama al método comer() de la superclase Animal

dormir(): llama al método dormir() de la superclase

emitirSonido(): return "Guau";

➤ Polimorfismo

< > Polimorfismo

¿Qué es el polimorfismo?:

En programación orientada a objetos, polimorfismo **es la capacidad que tienen los objetos de una clase en ofrecer respuesta distinta e independiente en función de los parámetros** (diferentes implementaciones) **utilizados durante su invocación** .
Dicho de otro modo el objeto como entidad puede contener valores de diferentes tipos durante la ejecución del programa.

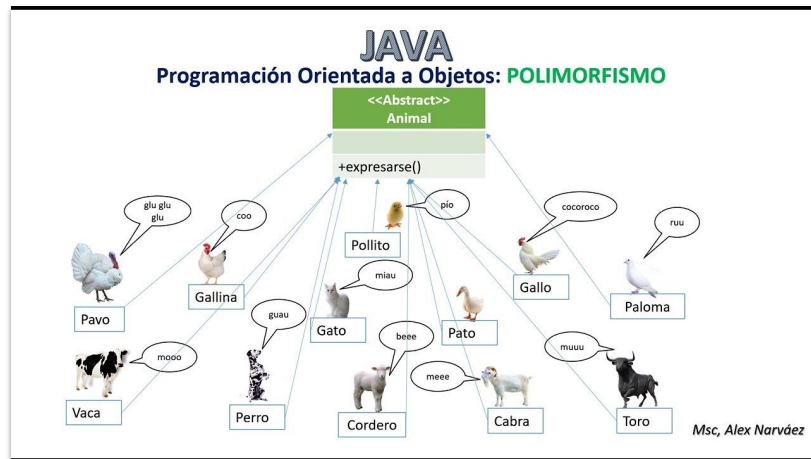
Basicamente, el polimorfismo en Java alude al **modo en que se pueden crear y utilizar dos o más métodos con el mismo nombre para ejecutar funciones diferentes** .



<> Polimorfismo

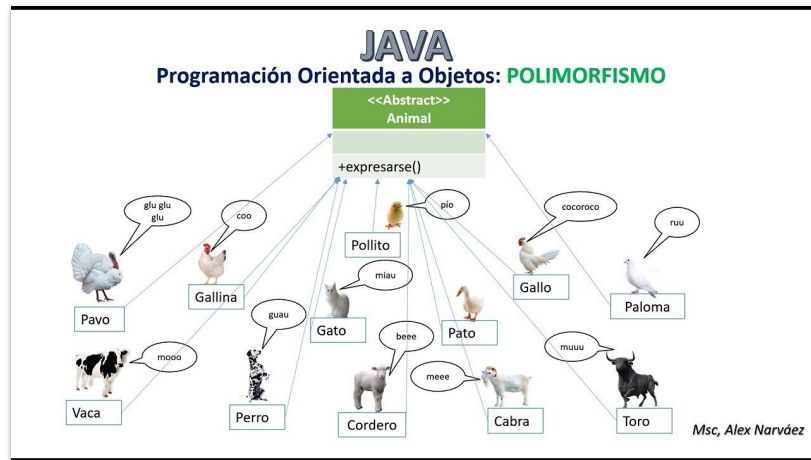
Veamos un ejemplo:

Imagina que tienes una mascota. Puede ser un perro, un gato o un pájaro. Cada uno de ellos tiene sus propias características y comportamientos específicos, como ladrar, maullar o volar. Ahora, piensa en cómo podrías representar todas estas mascotas en un programa de Java.



<> Polimorfismo

En lugar de crear una clase separada para cada tipo de mascota, podemos utilizar el polimorfismo para tratar a todas las mascotas de manera general. Esto significa que podemos tener una clase genérica llamada "Mascota" y luego crear subclases como "Perro", "Gato" y "Pájaro" que heredan de la clase "Mascota".

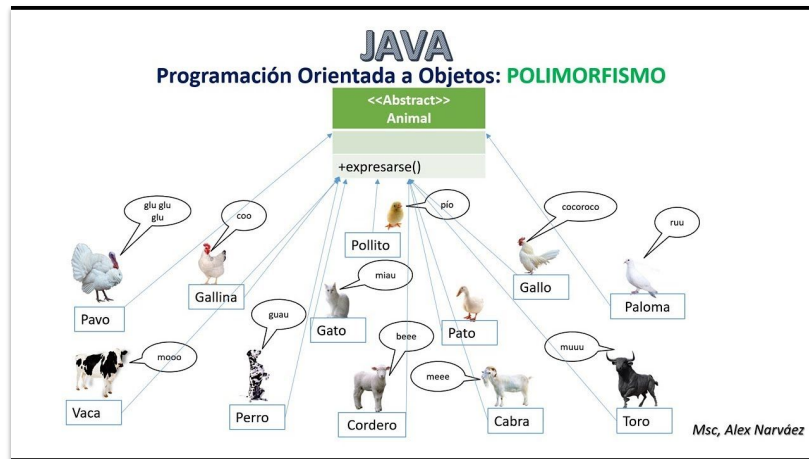


< > Polimorfismo

¿Pero, es igual a la clase Animal de los ejercicios?:

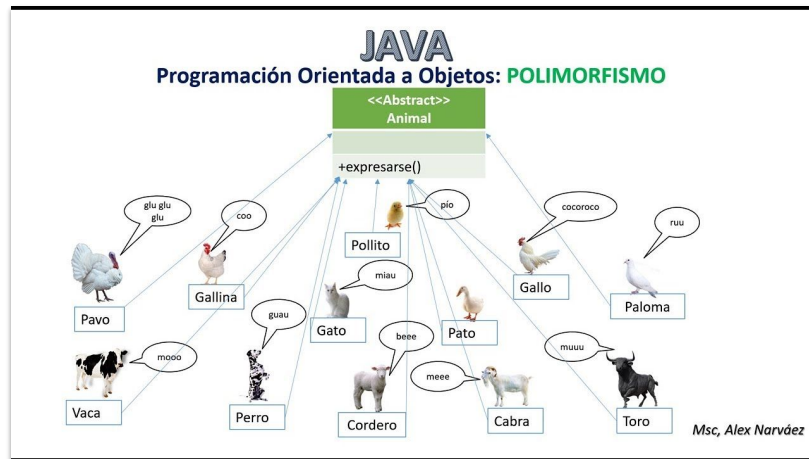
En realidad en este caso, por ejemplo, **la magia del polimorfismo ocurre cuando creamos un arreglo o una lista de mascotas**.

Podemos agregar instancias de diferentes subclases en la misma lista y **tratarlas como si fueran del tipo genérico "Mascota"**.



< > Polimorfismo

Cuando llamamos al método "hacerSonido()" en una mascota particular, Java se encarga de invocar el comportamiento correspondiente según el tipo de mascota real. Por ejemplo, si la mascota es un perro, se ladrará; si es un gato, se maullará; y así con todas las mascotas que agregues.

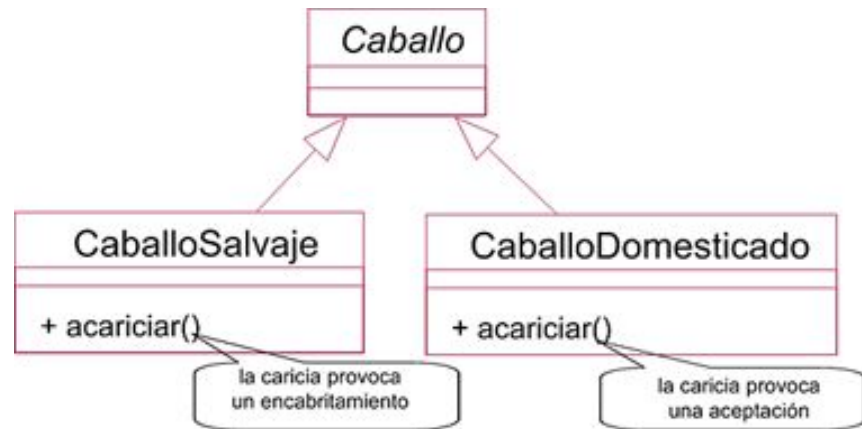


< > Polimorfismo mediante Interfaces

El método **acariciar** tiene un comportamiento diferente según si el caballo es una instancia de **CaballoSalvaje** o de **CaballoDomesticado**.

Si consideramos la clase **Caballo** en su totalidad, tenemos un conjunto de caballos que **reaccionan de distinta manera al activarse el método acariciar**.

Al tratar los objetos como instancias de la interfaz, podemos invocar los métodos definidos en la interfaz sin conocer la implementación específica de la clase.



Representando polimorfismo en un diagrama de clases



LIVE CODING

Apliquemos lo aprendido en el proyecto de Billetera Virtual:

- 1- Declararemos una superclase FormaDePago con el método void realizarPago():*
- 2- Luego se crean subclases TarjetaDeCrédito y Moneda que heredan de FormaDePago. La particularidad del pago con tarjeta es poder elegir cantidad de cuotas, mientras que Moneda puedes elegir el tipo de moneda.*
- 3- En la clase Billetera se debe declarar una referencia de tipo FormaDePago:
FormaDePago metodoPago;*

LIVE CODING

4- De esta manera podremos asignar el método de pago según se necesite:

```
metodoPago = new TarjetaDeCredito();
```

```
metodoPago = new Moneda();
```

Y utilizar polimorfismo al llamar: `metodoPago.realizarPago();`

Tiempo: 30 minutos



Momento:

Time-out!



5 -10 min.



➤ Clases abstractas

< > Clases abstractas

¿Qué son y para qué sirven?:

Las clases abstractas son una característica importante de la programación orientada a objetos en Java. Una clase abstracta es una clase que **no se puede instanciar** directamente y se utiliza como base para otras clases.

Habrà ocasiones en las cuales necesitemos crear una clase padre donde únicamente coloquemos la estructura de una abstracción, una estructura muy general, dejando que sean las clases hijas quienes definan los detalles. En estos casos haremos uso de las clases abstractas.



< > Clases abstractas

¿Qué son y para qué sirven?:

Usualmente las clases abstractas suelen ser las **superclases**, esto sucede porque creemos que la superclase o clase padre no debería poder instanciarse. Por ejemplo, si tenemos una clase `Animal`, el usuario no debería poder crear un `Animal`, sino que sólo debería poder instanciar objetos de las subclases (`Perro`, `Gato`, etc).

```
public abstract class Animal { }
```

Una clase abstracta es prácticamente idéntica a una clase convencional; las clases abstractas **pueden poseer atributos, métodos, constructores, etc** ... La principal diferencia entre una clase convencional y una clase abstracta es que **la clase abstracta debe poseer por lo menos un método abstracto**.



➤ Métodos abstractos

< > Métodos abstractos

¿Qué son y para qué se utilizan?:

Un método abstracto para Java es un método que **nunca va a ser ejecutado porque no tiene cuerpo**. Simplemente, un método abstracto referencia a otros métodos de las subclases.

¿Qué utilidad tiene un método abstracto? Podemos ver un método abstracto como una palanca que fuerza dos cosas: la primera, **que no se puedan crear objetos de una clase**. La segunda, que **todas las subclases sobrescriban el método** declarado como abstracto.



< > Métodos abstractos

Los métodos abstractos **deben ser declarados en clases abstractas y deben ser implementados por las clases que heredan** de la clase abstracta.

Los métodos abstractos se escriben **sin llaves {} y con ; al final** de la declaración.

Por ejemplo:

```
public abstract double area();
```

Un método se declara como abstracto porque en ese momento (en esa clase) no se conoce cómo va a ser su implementación.



< > Clases y métodos abstractos

Veamos un ejemplo:

En este caso la clase Figura posee una atributo, un constructor y un método, a partir de esta clase podré generar la n cantidad de figuras que necesite, ya sean cuadrados, rectangulos, triangulos, circulos etc...

Dentro de la clase encontramos el **método área** , método que se encuentra pensado para obtener el área de cualquier figura, sin embargo cómo sabemos todas las figuras poseen su propia fórmula matemática para calcular su área.

```
public class Figura {  
  
    private int numeroLados;  
  
    public Figura() {  
  
        this.numeroLados = 0;  
    }  
  
    public float area() {  
        return 0f;  
    }  
}
```



< > Clases y métodos abstractos

Si comenzamos a **heredar de la clase Figura**, todas las clases hijas tendrían que **sobreescribir el método área** e implementar su propia fórmula para así poder calcular su área. En estos casos en los que la clase hija siempre deba sobreescribir el método, es cuando debemos convertir al método convencional en un método abstracto, un método que defina qué hacer, pero no cómo se deba hacer. 😊

```
public abstract float area();
```



< > Clases y métodos abstractos

Ahora que el método área es un método abstracto la clase se convierte en una clase abstracta.

Es importante recordar que las clases abstractas pueden ser heredadas por la n cantidad de clases que necesitemos, pero **no pueden ser instanciadas** .

Al heredar de una clase abstracta **es obligatorio implementar todos sus métodos abstractos** , es decir debemos definir comportamiento, definir cómo se va a realizar la tarea.

```
public abstract class Figura {
```



LIVE CODING

Vamos a practicar la codificación para implementar clases y métodos abstractos:

1. *Declarar una clase abstracta CuentaDigital*
2. *Declarar un método abstracto verificarFondos()*
3. *Crear las subclases CuentaCL y CuentaUSD extendiendo de CuentaDigital.*
4. *Crear la lógica necesaria para implementar en cada método.*

Tiempo: 30 minutos

➤ Interfaces

< > Interfaces

¿Qué son y para qué se utilizan?:

Una interfaz **es una especie de plantilla** para la construcción de clases. Normalmente una interfaz **se compone de un conjunto de declaraciones de cabeceras de métodos** (sin implementar, de forma similar a un método abstracto) que especifican un protocolo de comportamiento para una o varias clases.

Una clase puede implementar una o varias interfaces . Por otro lado, una interfaz puede emplearse también para declarar constantes que luego puedan ser utilizadas por otras clases.

Las interfaces no pueden definir atributos salvo que estos sean estáticos o constantes; es decir, "**static**" o "**final**".



< > Interfaces

Entonces, ¿en qué se diferencian de una clase abstracta?

Una interfaz puede parecer similar a una clase abstracta, pero existen una serie de diferencias como las que mostramos a continuación:

- Todos los métodos de una interfaz se declaran **implícitamente** como abstractos y públicos.
- Una clase abstracta no puede implementar los métodos declarados como abstractos, **una interfaz no puede implementar ningún método** (ya que todos son abstractos).
- Una interfaz **no declara variables de instancia**.
- Una clase puede implementar varias interfaces, pero sólo puede tener una clase ascendiente directa.
- Una clase abstracta pertenece a una jerarquía de clases mientras que una interfaz no pertenece a una jerarquía de clases. En consecuencia, **clases sin relación de herencia pueden implementar la misma interfaz**.

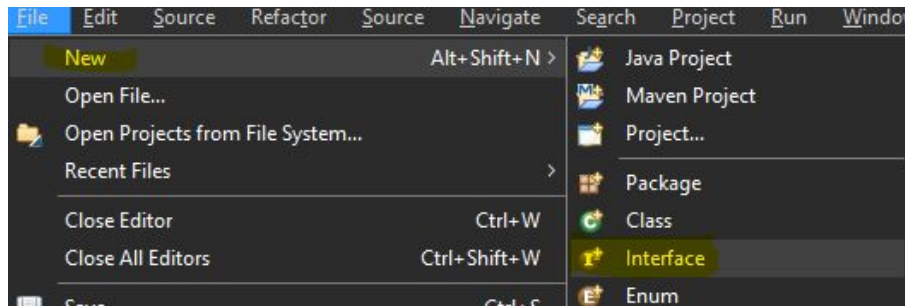


< > Interfaces

Declaración de una Interfaz

La declaración de una interfaz es similar a una clase, aunque emplea la palabra reservada **interface** en lugar de **class** y **no incluye ni la declaración de variables de instancia ni la implementación del cuerpo de los métodos** (sólo las cabeceras). La sintaxis de declaración de una interfaz es la siguiente:

```
public interface NombreInterfaz {  
    // Cuerpo de la interfaz ...  
}
```



< > Interfaces

Declaración de métodos y constantes:

Las constantes siempre son finales y se deben inicializar en la misma línea.

Los métodos son públicos y con declarar el tipo de retorno alcanza.

```
public interface MilInterfaz {  
    public final double CONSTANTE= 358,25;  
    public final int ENTERO= 125;  
    void put ( int dato );  
    int get (Object object);  
}
```



< > Interfaces

¿Cómo se implementan?

Para declarar una clase que implemente una interfaz es necesario utilizar la palabra reservada **implements** en la cabecera de declaración de la clase.

```
public class Clase implements NombreInterfaz {  
    // Cuerpo de la interfaz ...  
}
```



< > Interfaces

```
public interface Interfaz {  
    public void metodo();  
    public int sumar();  
}  
  
class Clase implements Interfaz {  
    @Override  
    public void metodo() {  
        System.out.println("Implementacion del método");  
    }  
  
    @Override  
    public int sumar() {  
        int suma = 2 + 2;  
        return suma;  
    }  
}
```

Veamos un ejemplo codificado:

Como podemos ver en la interfaz teníamos dos métodos sin cuerpo y al implementar la interfaz en nuestra clase, los sobrescribimos y les dimos una funcionalidad a dichos métodos.

LIVE CODING

Creando Interfaces: Vamos a modificar el proyecto BilleteraVirtual para hacer uso de las interfaces.

1. *Crear una interfaz **Moneda** con los métodos `getSimbolo()` y `getFactorConversion()`.*
2. *Crear una clase **Dolar** que implemente la interfaz Moneda.*
3. *Crear una clase **Euro** que implemente la interfaz Moneda.*
4. *Crear un método **convertir()** para la interfaz Moneda que permita convertir fondos de una moneda a otra.*

Tiempo: 30 minutos

› Polimorfismo mediante Interfaz

< > Polimorfismo mediante Interfaces

¿Cómo se implementa?:

Igual que con las clases, se puede conseguir el polimorfismo con las interfaces porque al implementar una interfaz podemos decir que **el objeto de la clase es un objeto de la interfaz** .

El polimorfismo entra en juego cuando utilizamos una interfaz como **tipo de referencia** en lugar de una clase concreta. Esto nos permite escribir código genérico que puede trabajar con diferentes clases que implementan la misma interfaz.



< > Polimorfismo mediante Interfaces

Veamos un ejemplo en código:

Las interfaces en Java nos permiten aplicar polimorfismo de una manera flexible.

La clave está en declarar referencias de la interfaz en lugar de la clase concreta.

Por ejemplo, podríamos tener una interfaz `Figura` con un método `obtenerArea()`:

```
public interface Figura {  
    public double obtenerArea();  
}
```



< > Polimorfismo mediante Interfaces

Luego hacemos que diferentes clases implementen la interfaz Figura y por consiguiente el método a implementar.

En este caso serán las clases Circulo y Rectangulo.

```
public class Circulo implements Figura {  
    @Override  
    public double obtenerArea() {
```

```
public class Rectangulo implements Figura{  
    @Override  
    public double obtenerArea() {
```



< > Polimorfismo mediante Interfaces

En la clase Main, vamos a crear una referencia a la interfaz para asignarle objetos Círculo o Rectángulo según sea necesario.

```
public static void main(String[] args) {  
  
    Figura figura; //referencia a la interfaz  
  
    figura = new Circulo();  
  
    figura = new Rectangulo();  
}
```



< > Polimorfismo mediante Interfaces

Al llamar a **figura.obtenerArea()** se ejecutará el método correspondiente al tipo de objeto real (Círculo o Rectángulo).

Esto es **polimorfismo mediante interfaces** , nos permite tratar objetos de distintas clases de manera uniforme a través de la interfaz en común.

La ventaja es que podemos agregar nuevas clases que implementen Figura y nuestro código seguirá funcionando sin necesidad de modificación.

```
public static void main(String[] args) {  
    Figura figura; //referencia a la interfaz  
  
    figura = new Circulo();  
    //llamamos al método de la clase Circulo  
    figura.obtenerArea();  
  
    figura = new Rectangulo();  
    //llamamos al método de la clase Rectangulo  
    figura.obtenerArea();  
}
```



LIVE CODING

Formas de Pago:

Un buen ejemplo de interfaz podría ser la forma de pago en una billetera electrónica. Esta forma de pago se aplica a cualquier medio de pago que se elija utilizar. Veamos un ejemplo:

*1- Crear una interfaz **FormaPago** con método `procesarPago()` e implementarla en las clases **TarjetaCredito**, **PayPal** y **Efectivo**. Probar polimorfismo pagando de distintas formas.*

Tiempo: 25 minutos



Ejercicio N° 1

Veterinaria 4.0

Veterinaria 4.0

Aplicando polimorfismo: 🙌

Como hemos visto, el polimorfismo se puede aplicar tanto mediante herencia como mediante interfaces. En este caso vamos a modificar el código de los ejercicios anteriores para practicar la implementación de Interfaces.

Consigna: 📝

1. Crear una interfaz **Animal** con métodos hacerRuido(), comer() y moverse().
2. **Implementar** en clases Perro, Pez, Gato y Ave.
3. Crear un **arreglo** de tipo “Animal” en el main y utilizar **polimorfismo** para mostrar la implementación de cada método.

Tiempo 🕒: 30 minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprendimos cómo se implementa la herencia en Java mediante extends.**
- ✓ **Aprendimos a utilizar `super()` para reutilizar constructores de la superclase.**
- ✓ **Analizamos el uso del polimorfismo con métodos sobrescritos e interfaces.**
- ✓ **Aplicamos clases y métodos abstractos para definir estructuras generales reutilizables.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

